

Apuntes de JavaScript

Para personas que no saben nada de programacion

BLOQUE 1 — Fundamentos

1	<u>¿Que es JavaScript?</u>	Para que sirve JS, donde se escribe, como ver resultados en consola (F12)
2	<u>Variables</u>	let, const, var — guardar datos. Reglas de nombre, camelCase
3	<u>Tipos de datos</u>	String, Number, Boolean, Array, Object, null, undefined. Template literals con `\${}`
4	<u>Condicionales</u>	if / else if / else. Operadores: ===, !==, >, <, &&, , !
5	<u>Funciones</u>	function, return, parametros. Funciones flecha =>. Diferencia entre definir y llamar

BLOQUE 2 — Estructuras de datos

6	<u>Arrays</u>	Listas de elementos. filter, find, forEach, map, sort, push, pop. Indices desde 0
7	<u>Objetos</u>	Clave:valor, objetos anidados, arrays dentro de objetos. Estructura del JSON
18	<u>reduce()</u>	Acumular arrays en un valor: sumar, contar, agrupar por categoria, normalizarDatos
24	<u>Map y Set</u>	Map: diccionario clave/valor avanzado. Set: lista sin duplicados, eliminar duplicados de array
25	<u>Object</u>	Object.keys/values/entries para recorrer objetos. Object.assign y spread para mezclarlos

BLOQUE 3 — DOM, eventos y HTML dinamico

8	<u>El DOM</u>	getElementById, querySelector. innerHTML, textContent. Crear elementos con createElement
9	<u>Eventos</u>	addEventListener: click, input, change, keydown. Tabla de todos los eventos comunes
17	<u>Tablas HTML dinamicas</u>	Generar tablas completas desde JS con innerHTML y createElement. Ejemplo con API de colores
17 b	<u>Imagenes desde API</u>	Mostrar imagenes cuya URL viene en el JSON. Ejemplo baraja de cartas. onerror, onload
26	<u>HTML de referencia</u>	Plantilla base, inputs, tablas, listas, imagenes, estilos inline, DOM — todo lo del examen

BLOQUE 4 — APIs y datos externos

10	fetch con .then()	Estructura base, dos .then(), .catch(). Fetch a fichero local o API real. find() sobre los datos
12	async / await	Alternativa moderna a .then(). try/catch en vez de .catch(). Color aleatorio hex
13	XMLHttpRequest	La forma clasica que piden algunos exámenes. open, onload, send, JSON.parse
14	localStorage	Guardar y leer datos sin servidor. JSON.stringify para guardar arrays. Patron de historial

BLOQUE 5 — Orientacion a Objetos (POO)

11	POO basica	class, constructor, this, metodos, getters/setters, static. Ejemplo Carrito completo
20	POO avanzada	Privado con #. Polimorfismo. instanceof. Clases abstractas con new.target. Mixins
20	Valores permitidos	Validar que un atributo solo acepta ciertos valores. includes() en constructor y setter. Object.freeze para enums

BLOQUE 6 — Strings, fechas y manejo de errores

15	Strings	Tabla de metodos: split, join, replace, trim, includes, padStart... Comprimir/descomprimir texto
16	Fechas	new Date, getFullYear/Month/Day. Validar DD/MM/AAAA. Años bisiestos. Dias entre fechas. Dia semana a texto
19	Errores comunes	Los fallos mas tipicos: = vs ===, null, .value, indices desde 0, fetch sin Live Server
22	try / catch / finally	Capturar errores sin romper el programa. throw new Error. Patron con fetch y async/await

BLOQUE 7 — Sintaxis moderna y herramientas

21	Sintaxis moderna	Parametros por defecto. Destructuring de arrays y objetos. Spread (...) y Rest (...)
23	Operadores modernos	?. acceder sin error si es null. ?? valor por defecto solo si null/undefined. Diferencia con
27	Extensiones VS Code	Live Server, Prettier, HTML CSS Support, snippets JS — todas offline. Atajos de teclado. Plan antes del examen
28	Resumen rapido	Tabla con todos los conceptos del documento en una pagina

1. ¿Que es JavaScript y para que sirve?

JavaScript (JS) es un lenguaje de programación que le da vida a las páginas web. Sin él, las webs solo serían texto estático. Con JavaScript puedes:

- Hacer que un botón haga algo al pulsarlo
- Mostrar u ocultar elementos de la página
- Comprobar si un formulario está bien rellenado
- Pedir datos a un servidor y mostrarlos sin recargar la página
- Crear juegos, animaciones, aplicaciones completas...

¿Donde se escribe?

Dentro del HTML, entre etiquetas `<script>...</script>`, o en un fichero aparte con extensión `.js` que se enlaza desde el HTML.

```
<!-- opcion 1: dentro del HTML -->
<script>
  console.log('Hola mundo')
</script>

<!-- opcion 2: fichero externo -->
<script src='mi-script.js'></script>
```

Para ver los resultados de tu código JS, abre las DevTools del navegador con F12 y ve a la pestaña Console.

2. Variables — guardar información

Una variable es como una caja con un nombre donde guardas un dato. Puedes meter cualquier cosa: un número, un texto, una lista...

```
let nombre = 'Ana'           // texto
let edad = 25                 // número
let esMayor = true           // verdadero/falso
let lista = [1, 2, 3]         // lista de cosas
```

Hay tres formas de declarar variables. Casi siempre usarás `let` o `const`:

Palabra	Para que se usa	Se puede cambiar?
---------	-----------------	-------------------

let	Variable normal que puede cambiar	Si
const	Variable que no va a cambiar nunca	No
var	Antigua, ya no se recomienda usar	Si (pero evitala)

```
let puntos = 0
puntos = 10           // bien, let permite cambiar el valor

const PI = 3.14
PI = 5                // ERROR: const no se puede cambiar
```

Reglas para nombrar variables

Sin espacios (usa camelCase: miVariable), sin tildes ni ñ, no pueden empezar por numero, distinguen mayusculas (nombre y Nombre son distintas).

3. Tipos de datos

Cada variable guarda un tipo de dato diferente. Los mas importantes:

Tipo	Ejemplo	Para que sirve
String (texto)	'Hola' o "Hola"	Nombres, frases, mensajes
Number (numero)	42 o 3.14	Edades, precios, cantidades
Boolean	true o false	Si/No, activado/desactivado
Array	[1, 2, 3]	Listas de cosas
Object	{ nombre: 'Ana' }	Agrupar datos relacionados
null	null	Vacio intencionadamente
undefined	undefined	Variable sin valor asignado

```
let nombre = 'Ana'           // String
let edad = 25                // Number
let activo = true            // Boolean
let notas = [8, 7, 9, 10]    // Array
let persona = {              // Object
  nombre: 'Ana',
  edad: 25
}
```

Puedes unir textos con el signo + o con template literals (comillas invertidas `):

```
let nombre = 'Ana'
let edad = 25

// con +
console.log('Hola ' + nombre + ', tienes ' + edad + ' anos')

// con template literal (mas comodo, usa acento grave `)
console.log(`Hola ${nombre}, tienes ${edad} anos`)
```

Resultado en consola:

```
Hola Ana, tienes 25 anos
Hola Ana, tienes 25 anos
```

4. Condicionales — tomar decisiones

Los condicionales le dicen al programa que haga una cosa u otra segun una condicion. El mas basico es if / else:

```
let edad = 20

if (edad >= 18) {
  console.log('Eres mayor de edad')
} else {
  console.log('Eres menor de edad')
}
```

Resultado en consola:

```
Eres mayor de edad
```

Puedes encadenar varias condiciones con else if:

```
let nota = 7

if (nota >= 9) {
  console.log('Sobresaliente')
} else if (nota >= 7) {
  console.log('Notable')
} else if (nota >= 5) {
  console.log('Aprobado')
} else {
```

```
console.log('Suspendido')
}
```

Resultado en consola:
Notable

Los operadores de comparacion que puedes usar:

Operador	Significa	Ejemplo
===	Igual (valor y tipo)	5 === 5 → true
!==	Distinto	5 !== 3 → true
>	Mayor que	10 > 5 → true
<	Menor que	3 < 8 → true
>=	Mayor o igual	5 >= 5 → true
<=	Menor o igual	4 <= 4 → true
&&	Y (ambas cosas)	edad > 18 && activo === true
	O (cualquiera)	esMayor tieneDNI
!	NO (niega)	!activo → lo invierte

⚠ Error comun

Usar = en vez de === dentro de un if. Un solo = asigna valor, tres === compara. Usa siempre === para comparar.

5. Funciones — bloques de codigo reutilizables

Una funcion es un bloque de codigo con nombre que puedes ejecutar cuando quieras, todas las veces que necesites. Evita repetir codigo.

```
// definir la funcion
function saludar(nombre) {
  console.log(`Hola, ${nombre}!`)
}

// llamarla (ejecutarla)
saludar('Ana')      // Hola, Ana!
saludar('Luis')     // Hola, Luis!
```

Resultado en consola:
Hola, Ana!

```
Hola, Luis!
```

Las funciones pueden devolver un resultado con return:

```
function sumar(a, b) {  
  return a + b      // devuelve el resultado  
}  
  
let resultado = sumar(3, 5)  
console.log(resultado) // 8
```

Resultado en consola:
8

Las funciones flecha son una forma mas corta de escribir funciones. Las veras mucho en ejemplos modernos:

```
// funcion normal  
function sumar(a, b) {  
  return a + b  
}  
  
// funcion flecha (hace lo mismo)  
const sumar = (a, b) => {  
  return a + b  
}  
  
// version ultra corta (si solo tiene una linea)  
const sumar = (a, b) => a + b
```

Resultado en consola:
Las tres versiones devuelven: 8 (para sumar(3, 5))

6. Arrays — listas de cosas

Un array es una lista ordenada de elementos. Cada elemento tiene una posicion llamada indice que empieza en 0.

```
let frutas = ['manzana', 'pera', 'naranja']  
//           [0]         [1]         [2]
```

```
console.log(frutas[0])    // 'manzana'
console.log(frutas[2])    // 'naranja'
console.log(frutas.length) // 3 (numero de elementos)
```

Resultado en consola:

```
manzana
naranja
3
```

Metodos mas usados con arrays:

```
let numeros = [5, 2, 8, 1, 9]

// recorrer todos los elementos
numeros.forEach(n => console.log(n))

// filtrar los que cumplen una condicion
let mayores = numeros.filter(n => n > 4)    // [5, 8, 9]

// buscar uno concreto
let encontrado = numeros.find(n => n === 8) // 8

// transformar cada elemento
let dobles = numeros.map(n => n * 2)    // [10, 4, 16, 2, 18]

// ordenar
numeros.sort((a, b) => a - b)    // [1, 2, 5, 8, 9]

// añadir al final
numeros.push(7)

// quitar el ultimo
numeros.pop()
```

Resultado en consola:

```
forEach → 5 2 8 1 9 (uno por linea)
filter  → [ 5, 8, 9 ]
find     → 8
map      → [ 10, 4, 16, 2, 18 ]
sort     → [ 1, 2, 5, 8, 9 ]
```



Los indices empiezan en 0

Este es uno de los errores mas tipicos. Si tienes 3 elementos, los indices son 0, 1 y 2. El indice 3 no existe y devolveria undefined.

7. Objetos — agrupar informacion

Un objeto agrupa datos relacionados bajo un mismo nombre. Cada dato tiene una clave (nombre) y un valor.

```
let persona = {
  nombre: 'Ana',
  edad: 25,
  activa: true
}

// acceder a las propiedades
console.log(persona.nombre) // 'Ana'
console.log(persona.edad)   // 25

// modificar una propiedad
persona.edad = 26

// anadir una propiedad nueva
persona.ciudad = 'Madrid'
```

Resultado en consola:

Ana
25

Los objetos pueden tener objetos y arrays dentro (anidados):

```
let alumno = {
  nombre: 'Luis',
  notas: [8, 7, 9],           // array dentro del objeto
  direccion: {               // objeto dentro del objeto
    ciudad: 'Barcelona',
    calle: 'Gran Via'
  }
}

console.log(alumno.notas[0]) // 8
console.log(alumno.direccion.ciudad) // 'Barcelona'
```

Resultado en consola:

8
Barcelona

JSON (JavaScript Object Notation) es simplemente objetos y arrays de JS escritos en un fichero de texto. Es el formato mas usado para intercambiar datos:

```
// fichero datos.json
[
  { "nombre": "Ana", "edad": 25 },
  { "nombre": "Luis", "edad": 30 }
]
```

8. El DOM — conectar JS con el HTML

El DOM (Document Object Model) es la forma que tiene JavaScript de acceder y modificar el HTML de la página. Piensalo como un mapa del HTML que JS puede leer y editar.

Lo más habitual es seleccionar un elemento del HTML por su id y luego modificarlo:

```
// HTML: <p id="mensaje">Hola</p>

// seleccionar el elemento
let parrafo = document.getElementById('mensaje')

// cambiar su texto
parrafo.textContent = 'Adios'

// cambiar su HTML interior
parrafo.innerHTML = '<strong>Adios</strong>'

// cambiar su estilo
parrafo.style.color = 'red'

// ocultarlo
parrafo.style.display = 'none'
```

Otras formas de seleccionar elementos:

<code>document.getElementById('miId')</code>	<code>// por id (devuelve uno)</code>
<code>document.querySelector('.miClase')</code> (devuelve el primero)	<code>// por clase o selector CSS</code>
<code>document.querySelectorAll('p')</code> (devuelve lista)	<code>// todos los parrafos</code>

9. Eventos — reaccionar a acciones del usuario

Un evento es algo que ocurre en la página: un click, una tecla pulsada, el ratón moviéndose... Puedes decirle a JS que ejecute una función cuando ocurra un evento.

```
// HTML: <button id="btn">Pulsar</button>

let boton = document.getElementById('btn')

boton.addEventListener('click', () => {
  alert('Has pulsado el boton!')
})
```

Eventos mas habituales:

Evento	Cuando se dispara
click	Al hacer click en el elemento
input	Al escribir en un campo de texto
change	Al cambiar el valor de un select o checkbox
submit	Al enviar un formulario
keydown	Al pulsar una tecla
mouseover	Al pasar el raton por encima
load	Cuando la pagina termina de cargarse

Ejemplo tipico: leer lo que el usuario escribe en un input y mostrarlo:

```
// HTML:
// <input type="text" id="nombre" />
// <p id="saludo"></p>

let input = document.getElementById('nombre')
let saludo = document.getElementById('saludo')

input.addEventListener('input', () => {
  let texto = input.value // .value lee lo que hay en el
  input
  saludo.textContent = `Hola, ${texto}!`
})
```

10. Fetch — pedir datos a un fichero o API

`fetch()` sirve para pedir datos a un fichero JSON local o a una API en internet. Es asincrono, lo que significa que no bloquea la pagina mientras espera la respuesta.

La estructura siempre es la misma:

```
fetch('ruta/fichero.json')           // 1. pides el fichero
  .then(respuesta => respuesta.json()) // 2. conviertes a JSON
  .then(datos => {                     // 3. trabajas con los datos
    console.log(datos)
  })
  .catch(error => {                   // 4. si algo falla
    console.error(error)
  })
```

Ejemplo completo: buscar una palabra en un JSON local y mostrar su definicion:

```
let boton = document.getElementById('buscar')
let input = document.getElementById('palabra')
let resultado = document.getElementById('resultado')

boton.addEventListener('click', () => {
  let termino = input.value.trim().toLowerCase()

  if (!termino) {
    resultado.innerHTML = '<p>Escribe una palabra</p>'
    return
  }

  fetch('data/diccionario.json')
    .then(res => res.json())
    .then(datos => {

      // buscar la palabra en el array
      let entrada = datos.find(item =>
        item.word.toLowerCase() === termino
      )

      if (!entrada) {
        resultado.innerHTML = '<p>Palabra no encontrada</p>'
        return
      }

      // mostrar el resultado
      resultado.innerHTML = `<h2>${entrada.word}</h2>`
      entrada.meanings[0].definitions.forEach(d => {
```

```
        resultado.innerHTML += `<p>${d.definition}</p>`
    })
})
.catch(err => {
    resultado.innerHTML = 'Error al cargar los datos'
})
})
```

📌 ¿Por que dos .then()?

fetch() devuelve una 'promesa' de respuesta, no los datos directamente. El primer .then() abre esa respuesta y lee el JSON. El segundo ya trabaja con los datos de verdad.

11. Orientacion a Objetos (POO)

La Programacion Orientada a Objetos (POO) es una forma de organizar el codigo agrupando datos y funciones relacionados en una entidad llamada clase. Una clase es como un molde, y los objetos son las piezas que fabricas con ese molde.

📌 Analogia

Una clase es como el plano de una casa. Con un plano puedes construir muchas casas (objetos). Todas siguen el mismo plano pero cada una tiene sus propios datos (color, propietario...).

Crear una clase

Se define con la palabra class. Dentro tiene un constructor (se ejecuta al crear el objeto) y metodos (funciones propias de la clase):

```
class Persona {
    // constructor: se ejecuta al crear el objeto
    constructor(nombre, edad) {
        this.nombre = nombre // this.X guarda el dato en el objeto
        this.edad = edad
    }

    // metodo: funcion propia de la clase
    saludar() {
        console.log(`Hola, soy ${this.nombre} y tengo ${this.edad} anos`)
    }

    esMayor() {
        return this.edad >= 18
    }
}
```

```
}  
}
```

Crear objetos (instanciar)

Para crear un objeto a partir de una clase usas new. Puedes crear todos los que quieras:

```
let ana = new Persona('Ana', 25)  
let luis = new Persona('Luis', 16)  
  
ana.saludar()           // Hola, soy Ana y tengo 25 anos  
luis.saludar()          // Hola, soy Luis y tengo 16 anos  
  
console.log(ana.esMayor()) // true  
console.log(luis.esMayor()) // false  
  
// acceder a las propiedades directamente  
console.log(ana.nombre)    // 'Ana'  
console.log(luis.edad)     // 16
```

Resultado en consola:

```
Hola, soy Ana y tengo 25 anos  
Hola, soy Luis y tengo 16 anos  
true  
false  
Ana  
16
```

Herencia — una clase basada en otra

Una clase puede heredar todo de otra clase y añadirle cosas nuevas. Se usa extends. Así no repites código:

```
class Animal {  
  constructor(nombre) {  
    this.nombre = nombre  
  }  
  
  hablar() {  
    console.log(`${this.nombre} hace un sonido`)  
  }  
}  
  
// Perro hereda todo de Animal  
class Perro extends Animal {  
  constructor(nombre, raza) {  

```

```

    super(nombre)    // llama al constructor del padre (Animal)
    this.raza = raza
  }

  // sobrescribe el metodo hablar
  hablar() {
    console.log(`${this.nombre} ladra: Guau!`)
  }

  // metodo propio de Perro
  info() {
    console.log(`${this.nombre} es un ${this.raza}`)
  }
}

let miPerro = new Perro('Rex', 'Pastor Aleman')
miPerro.hablar()    // Rex ladra: Guau!
miPerro.info()      // Rex es un Pastor Aleman

```

Resultado en consola:

```

Rex ladra: Guau!
Rex es un Pastor Aleman

```

super()

Cuando una clase hereda de otra y tiene constructor, debes llamar a `super()` al principio pasandole los parametros del padre. Si no lo haces, JS da error.

Getters y setters

Los getters y setters permiten controlar como se leen y modifican las propiedades de un objeto. Son utiles para validar datos antes de guardarlos:

```

class Producto {
  constructor(nombre, precio) {
    this.nombre = nombre
    this._precio = precio    // _ indica que es 'privado' por
    convencion
  }

  // getter: se usa como propiedad, no como funcion
  get precio() {
    return `${this._precio} euros`
  }

  // setter: valida antes de guardar
  set precio(valor) {
    if (valor < 0) {
      console.log('El precio no puede ser negativo')
    } else {
      this._precio = valor
    }
  }
}

```

```

    }
  }
}

let p = new Producto('Camisa', 20)
console.log(p.precio)      // '20 euros'   (getter)
p.precio = 25              // setter
p.precio = -5              // 'El precio no puede ser negativo'

```

Resultado en consola:

```

20 euros
25 euros
El precio no puede ser negativo

```

Metodos y propiedades estaticas

Los metodos estaticos pertenecen a la clase en si, no a cada objeto. Se llaman directamente desde la clase sin crear un objeto:

```

class Calculadora {
  static sumar(a, b) {
    return a + b
  }

  static restar(a, b) {
    return a - b
  }
}

// no necesitas new, lo llamas directo desde la clase
console.log(Calculadora.sumar(3, 5)) // 8
console.log(Calculadora.restar(10, 4)) // 6

```

Resultado en consola:

```

8
6

```

Ejemplo practico completo

Una clase Carrito de la compra que gestiona productos:

```

class Carrito {
  constructor() {
    this.productos = [] // empieza vacio
  }
}

```



```

    anadir(producto, precio) {
      this.productos.push({ producto, precio })
      console.log(`${producto} anadido`)
    }

    eliminar(producto) {
      this.productos = this.productos.filter(p => p.producto !==
producto)
    }

    get total() {
      return this.productos.reduce((suma, p) => suma + p.precio, 0)
    }

    mostrar() {
      this.productos.forEach(p => {
        console.log(`${p.producto}: ${p.precio} euros`)
      })
      console.log(`Total: ${this.total} euros`)
    }
  }

  let miCarrito = new Carrito()
  miCarrito.anadir('Camisa', 20)
  miCarrito.anadir('Pantalon', 35)
  miCarrito.anadir('Zapatos', 50)
  miCarrito.mostrar()
  // Camisa: 20 euros
  // Pantalon: 35 euros
  // Zapatos: 50 euros
  // Total: 105 euros
  miCarrito.eliminar('Camisa')
  console.log(miCarrito.total)    // 85

```

Resultado en consola:

```

Camisa anadido
Pantalon anadido
Zapatos anadido
Camisa: 20 euros
Pantalon: 35 euros
Zapatos: 50 euros
Total: 105 euros
85

```

Resumen de los conceptos clave de POO:

Concepto	Para que sirve	Palabra clave
Clase	Molde para crear objetos	class
Constructor	Se ejecuta al crear el objeto, inicializa datos	constructor()
this	Hace referencia al propio objeto	this.propiedad
Instancia	Objeto creado a partir de una clase	new NombreClase()

Metodo	Funcion propia de la clase	nombreMetodo() { }
Herencia	Una clase hereda de otra	extends
super()	Llama al constructor del padre	super(params)
Getter	Leer una propiedad con logica	get nombre() { }
Setter	Modificar una propiedad con validacion	set nombre(v) { }
Estatico	Metodo de la clase, no del objeto	static

12. async/await — otra forma de hacer fetch

async/await es una forma mas moderna y legible de escribir codigo asincrono. Hace lo mismo que .then() pero parece codigo normal, sin encadenar funciones.

Comparacion directa entre las dos formas:

```
// con .then() (forma clasica)
fetch('datos.json')
  .then(res => res.json())
  .then(data => {
    console.log(data)
  })
  .catch(err => console.error(err))

// con async/await (forma moderna, hace lo mismo)
async function cargarDatos() {
  try {
    const res = await fetch('datos.json') // espera la respuesta
    const data = await res.json()        // espera abrir el JSON
    console.log(data)
  } catch (err) {
    console.error(err)
  }
}

cargarDatos() // la llamas normal
```

await solo funciona dentro de async

No puedes usar await fuera de una funcion marcada con async. Si lo intentas, JS te dara un error de sintaxis.

Ejemplo real: peticion a una API con async/await (examen de colores):

```

async function obtenerColor(hex) {
  try {
    const url = `https://www.thecolorapi.com/id?hex=${hex}`
    const res = await fetch(url)

    if (!res.ok) throw new Error('Error en la petición')

    const data = await res.json()

    // datos que devuelve la API de colores
    console.log(data.name.value)           // nombre del color
    console.log(data.hex.value)           // '#FF5733'
    console.log(data.contrast.value)       // color de texto
    recomendado

    return data
  } catch (err) {
    console.error('Fallo: ' + err.message)
  }
}

obtenerColor('FF5733')

```

Generar un color hexadecimal aleatorio (util en exámenes):

```

function colorAleatorio() {
  // genera un numero aleatorio entre 0 y 16777215 (= 0xFFFFFF)
  let num = Math.floor(Math.random() * 16777215)
  // lo convierte a hexadecimal y rellena con ceros si hace falta
  return num.toString(16).padStart(6, '0').toUpperCase()
}

console.log(colorAleatorio()) // por ejemplo: 'A3F2B1'

```

Resultado en consola:

```

6EA872  (cambia cada vez, es aleatorio)
7BF84E
52E218

```

13. XMLHttpRequest — la forma antigua de fetch

XMLHttpRequest (XHR) es la forma original de hacer peticiones HTTP en JavaScript, antes de que existiera fetch. Algunos exámenes la piden explícitamente, así que hay que conocerla.

Comparacion con fetch:

	fetch	XMLHttpRequest
Sintaxis	Moderna y limpia	Mas verbosa
Promesas	Si, usa .then()	No, usa eventos (onload)
async/await	Compatible	No directamente
Cuando usarla	Siempre que puedas	Cuando el examen lo pida

Estructura basica de XMLHttpRequest:

```
const xhr = new XMLHttpRequest() // 1. crear el objeto
xhr.open('GET', 'https://api.ejemplo.com/datos') // 2. configurar
xhr.onload = function() { // 3. que hacer cuando llega
  if (xhr.status === 200) { // si va bien (codigo 200)
    const data = JSON.parse(xhr.responseText) // convertir a objeto
    console.log(data)
  } else {
    console.error('Error: ' + xhr.status)
  }
}
xhr.onerror = function() { // 4. si hay error de red
  console.error('Error de conexion')
}
xhr.send() // 5. enviar la peticion
```

Ejemplo real del examen — API de edad por nombre (agify.io):

```
function consultarEdad(nombre) {
  const xhr = new XMLHttpRequest()
  const url = `https://api.agify.io/?name=${nombre}`

  xhr.open('GET', url)

  xhr.onload = function() {
    if (xhr.status === 200) {
      const data = JSON.parse(xhr.responseText)
      // data.name → el nombre
      // data.age → la edad estimada
    }
  }
}
```

```
        console.log(`${data.name} tiene aprox. ${data.age} anos`)  
      }  
    }  
    xhr.send()  
  }  
  consultarEdad('Carlos')
```

📌 JSON.parse vs .json()

Con fetch usas `res.json()` para convertir la respuesta. Con XHR usas `JSON.parse(xhr.responseText)` porque te llega como texto plano, no como promesa.

14. localStorage — guardar datos en el navegador

`localStorage` permite guardar datos en el navegador del usuario de forma permanente. Los datos persisten aunque se cierre el navegador o se recargue la página.

📌 Solo guarda texto

`localStorage` solo puede guardar strings. Para guardar objetos o arrays tienes que convertirlos a JSON con `JSON.stringify()` y al leerlos convertirlos de vuelta con `JSON.parse()`.

Operaciones básicas:

```
// guardar un dato simple  
localStorage.setItem('nombre', 'Ana')  
  
// leer un dato  
let nombre = localStorage.getItem('nombre') // 'Ana'  
  
// eliminar un dato  
localStorage.removeItem('nombre')  
  
// borrar todo el localStorage  
localStorage.clear()
```

Guardar y leer arrays u objetos (lo más habitual en exámenes):

```
// guardar un array
let resultados = [
  { nombre: 'Ana', edad: 25 },
  { nombre: 'Luis', edad: 30 }
]
localStorage.setItem('resultados', JSON.stringify(resultados))

// leer el array
let guardados = localStorage.getItem('resultados')
let array = guardados ? JSON.parse(guardados) : []
// el ? [] es por si no hay nada guardado todavia
```

Patron tipico de examen: guardar cada resultado y mostrarlo en tabla al cargar:

```
// al hacer una consulta, guardar el resultado
function guardarResultado(nombre, edad) {
  let datos = JSON.parse(localStorage.getItem('consultas') || '[]')
  datos.push({ nombre, edad }) // anadir nuevo
  localStorage.setItem('consultas', JSON.stringify(datos)) //
  guardar
  mostrarTabla() // actualizar tabla
}

// al cargar la pagina, recuperar y mostrar
function mostrarTabla() {
  let datos = JSON.parse(localStorage.getItem('consultas') || '[]')
  let tabla = document.getElementById('tabla')

  tabla.innerHTML = '<tr><th>Nombre</th><th>Edad estimada</th></tr>'

  datos.forEach(item => {
    tabla.innerHTML += `<tr><td>${item.nombre}</td><td>${
item.edad}</td></tr>`
  })
}

// ejecutar al cargar la pagina
window.onload = mostrarTabla
```

15. Strings — manipular texto

Los strings (cadenas de texto) tienen muchos metodos utiles. Estos son los que mas aparecen en exámenes:

Metodo	Que hace	Ejemplo
--------	----------	---------

length	Longitud del texto	'hola'.length → 4
toUpperCase()	Todo a mayusculas	'hola'.toUpperCase() → 'HOLA'
toLowerCase()	Todo a minusculas	'HOLA'.toLowerCase() → 'hola'
trim()	Quita espacios al inicio y fin	' hola '.trim() → 'hola'
includes()	Comprueba si contiene algo	'hola mundo'.includes('mundo') → true
startsWith()	Empieza por...	'hola'.startsWith('ho') → true
endsWith()	Acaba por...	'hola'.endsWith('la') → true
indexOf()	Posicion de un caracter	'hola'.indexOf('l') → 2
slice(i, f)	Trozo del texto	'hola'.slice(1, 3) → 'ol'
split(sep)	Divide en array	'a,b,c'.split(',') → ['a','b','c']
replace(a, b)	Sustituye la primera vez	'hola'.replace('l','r') → 'hora'
replaceAll(a, b)	Sustituye todas las veces	'lol'.replaceAll('l','r') → 'ror'
charAt(i)	Caracter en posicion i	'hola'.charAt(0) → 'h'
padStart(n, c)	Rellena por la izquierda	'5'.padStart(3,'0') → '005'
padEnd(n, c)	Rellena por la derecha	'5'.padEnd(3,'0') → '500'

Ejemplo del examen de compresion de textos:

```
// comprimir 'aaabccddde' → 'a3b1c2d4e1'
function comprimir(texto) {
  let resultado = ''
  let i = 0

  while (i < texto.length) {
    let letra = texto[i]
    let contador = 0

    // contar cuantas veces se repite seguida
    while (i < texto.length && texto[i] === letra) {
      contador++
      i++
    }
    resultado += letra + contador
  }
  return resultado
}

// descomprimir 'a3b1c2' → 'aaabcc'
function descomprimir(texto) {
  let resultado = ''
  let i = 0

  while (i < texto.length) {
    let letra = texto[i]
    let num = parseInt(texto[i + 1])
```

```

        resultado += letra.repeat(num)
        i += 2
    }
    return resultado
}

console.log(comprimir('aaabccddde')) // 'a3b1c2d4e1'
console.log(descomprimir('a3b1c2'))  // 'aaabcc'

```

Resultado en consola:
a3b1c2d4e1
aaabcc

Validar que un string solo tiene letras (util para el examen de compresion):

```

function soloLetras(texto) {
    return /^[a-zA-Z]+$/.test(texto) // expresion regular
}

soloLetras('hola') // true
soloLetras('ho3la') // false
soloLetras('') // false

```

Resultado en consola:
true
false
false

16. Fechas — trabajar con Date

El objeto Date de JavaScript permite crear, comparar y manipular fechas. Es un tema que aparece en exámenes con validaciones de formato y calculos entre fechas.

Crear fechas

```

let hoy = new Date() // fecha y hora actuales
let fecha = new Date(2024, 0, 15) // 15 enero 2024 (mes empieza en 0!)
let fecha2 = new Date('2024-01-15') // desde string ISO

// OJO: los meses van de 0 a 11
// 0=enero, 1=febrero, ..., 11=diciembre

```


Leer partes de una fecha

```
let f = new Date(2024, 2, 15) // 15 de marzo de 2024

f.getFullYear()    // 2024
f.getMonth()       // 2 (marzo, OJO: empieza en 0)
f.getDate()        // 15 (dia del mes)
f.getDay()         // dia de la semana (0=domingo,
1=lunes...6=sabado)
f.getTime()        // milisegundos desde 1/1/1970 (util para
comparar)
```

Resultado en consola:

```
2024
2
15
5 (viernes)
1710460800000
```

Convertir dia de semana a texto

```
function diaSemana(fecha) {
  const dias = ['Domingo', 'Lunes', 'Martes', 'Miercoles',
    'Jueves', 'Viernes', 'Sabado']
  return dias[fecha.getDay()]
}

let f = new Date(2024, 2, 15) // viernes 15 marzo 2024
console.log(diaSemana(f))    // 'Viernes'
```

Resultado en consola:

```
Viernes
```

Validar fecha en formato DD/MM/AAAA

```
function validarFecha(texto) {
  // comprobar formato DD/MM/AAAA
  const partes = texto.split('/')
  if (partes.length !== 3) return false

  const dia = parseInt(partes[0])
  const mes = parseInt(partes[1])
  const anyo = parseInt(partes[2])

  // rangos basicos
  if (mes < 1 || mes > 12) return false
  if (dia < 1) return false

  // dias maximos segun el mes
```

```

const diasMes = [0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

// año bisiesto: febrero tiene 29 días
if (anyo % 4 === 0 && (anyo % 100 !== 0 || anyo % 400 === 0)) {
  diasMes[2] = 29
}

if (dia > diasMes[mes]) return false

return true
}

validarFecha('31/02/2024') // false (febrero no tiene 31 días)
validarFecha('29/02/2024') // true (2024 es bisiesto)
validarFecha('15/03/2024') // true

```

Resultado en consola:

```

false
true
true

```

Calcular días entre dos fechas

```

function diasEntre(fecha1, fecha2) {
  // getTime() devuelve milisegundos
  const diff = fecha2.getTime() - fecha1.getTime()
  // convertir milisegundos a días
  return Math.abs(Math.round(diff / (1000 * 60 * 60 * 24)))
}

let f1 = new Date(2024, 0, 1) // 1 enero
let f2 = new Date(2024, 0, 15) // 15 enero
console.log(diasEntre(f1, f2)) // 14

```

Resultado en consola:

```

14

```

Ejemplo completo del examen de fechas

```

function procesarFechas(texto1, texto2) {
  // validar las dos fechas
  if (!validarFecha(texto1)) {
    console.log('Fecha 1 incorrecta'); return
  }
  if (!validarFecha(texto2)) {
    console.log('Fecha 2 incorrecta'); return
  }

  // convertir strings a objetos Date
  const p1 = texto1.split('/')

```

```

    const p2 = texto2.split('/')
    const f1 = new Date(p1[2], p1[1]-1, p1[0]) // mes-1 porque empieza
    en 0
    const f2 = new Date(p2[2], p2[1]-1, p2[0])

    // comprobar que f2 es posterior a f1
    if (f2 <= f1) {
        console.log('La segunda fecha debe ser posterior'); return
    }

    console.log(`Fecha 1: ${diaSemana(f1)}`)
    console.log(`Fecha 2: ${diaSemana(f2)}`)
    console.log(`Dias entre fechas: ${diasEntre(f1, f2)}`)
}

procesarFechas('01/01/2024', '15/03/2024')

```

Resultado en consola:

```

Fecha 1: Lunes
Fecha 2: Viernes
Dias entre fechas: 74

```

17. Tablas HTML dinamicas desde JavaScript

Uno de los ejercicios mas habituales en exámenes es generar una tabla HTML completamente desde JS sin tener nada escrito en el HTML. Hay dos formas: con innerHTML o con createElement.

Con innerHTML (mas rapido de escribir)

```

function crearTabla(datos) {
    let html = '<table border="1">'

    // cabecera
    html += '<tr><th>Nombre</th><th>Edad</th></tr>'

    // filas de datos
    datos.forEach(item => {
        html += `<tr><td>${item.nombre}</td><td>${item.edad}</td></tr>`
    })

    html += '</table>'
    document.getElementById('contenedor').innerHTML = html
}

```

Con createElement (mas correcto)

```

function crearTabla(datos) {

```

```

const tabla = document.createElement('table')

// fila de cabecera
const cabecera = document.createElement('tr')
['Nombre', 'Edad'].forEach(texto => {
  const th = document.createElement('th')
  th.textContent = texto
  cabecera.appendChild(th)
})
tabla.appendChild(cabecera)

// filas de datos
datos.forEach(item => {
  const fila = document.createElement('tr')

  const tdNombre = document.createElement('td')
  tdNombre.textContent = item.nombre

  const tdEdad = document.createElement('td')
  tdEdad.textContent = item.edad

  fila.appendChild(tdNombre)
  fila.appendChild(tdEdad)
  tabla.appendChild(fila)
})

// insertar en el DOM, vaciando primero el contenedor
const contenedor = document.getElementById('contenedor')
contenedor.innerHTML = ''
contenedor.appendChild(tabla)
}

```

Ejemplo del examen de colores — tabla con celdas de colores aleatorios:

```

async function generarTablaColores(filas, columnas) {
  const tabla = document.createElement('table')

  for (let f = 0; f < filas; f++) {
    const fila = document.createElement('tr')

    for (let c = 0; c < columnas; c++) {
      const hex = colorAleatorio() // funcion de antes
      const res = await fetch(`https://www.thecolorapi.com/id?hex=${hex}`)
      const data = await res.json()

      const celda = document.createElement('td')
      celda.textContent = '#' + hex
      celda.style.backgroundColor = '#' + hex
      celda.style.color = data.contrast.value // color de texto
      recomendado
      celda.style.padding = '10px'
      celda.style.textAlign = 'center'

      fila.appendChild(celda)
    }
  }
}

```

```

        tabla.appendChild(fila)
    }

    document.getElementById('contenedor').innerHTML = ''
    document.getElementById('contenedor').appendChild(tabla)
}

// leer los inputs del usuario y generar la tabla
document.getElementById('btn').addEventListener('click', () => {
    const filas = parseInt(document.getElementById('filas').value)
    const cols = parseInt(document.getElementById('columnas').value)
    generarTablaColores(filas, cols)
})

```

17b. Mostrar imágenes desde una API

Muchas APIs devuelven URLs de imágenes dentro del JSON. El proceso es siempre el mismo: hacer fetch, extraer la URL del campo correspondiente, crear un elemento `<img>` y asignarle el `src`.

Mostrar una imagen

Lo más básico: fetch a la API, sacar la URL de imagen del JSON, crear un `<img>` y meterlo en el DOM.

```

fetch('https://api.ejemplo.com/foto')
    .then(res => res.json())
    .then(data => {

        // la API devuelve algo como: { image: 'https://...jpg', title:
        'Foto' }
        let img = document.createElement('img')
        img.src = data.image // la URL viene del JSON
        img.alt = data.title // texto alternativo
        img.style.width = '200px' // tamaño opcional

        document.getElementById('contenedor').appendChild(img)
    })

```

Ejemplo real — API de la baraja de cartas (examen)

La API de la baraja devuelve cada carta con una URL de imagen. Primero obtienes el `deck_id` y luego robas cartas:

```

// la API devuelve esto al robar una carta:
// {
//   cards: [{
//     image: 'https://deckofcardsapi.com/static/img/KH.png',
//     value: 'KING',
//     suit: 'HEARTS',
//     code: 'KH'
//   }],
//   remaining: 51
// }

let deckId = null

// paso 1: obtener el id de la baraja al cargar la pagina
window.onload = function() {
  fetch('https://deckofcardsapi.com/api/deck/new/')
    .then(res => res.json())
    .then(data => {
      deckId = data.deck_id
      console.log('Baraja creada:', deckId)
    })
}

// paso 2: robar una carta al pulsar el boton
document.getElementById('btn').addEventListener('click', () => {
  fetch(`https://deckofcardsapi.com/api/deck/${deckId}/draw/?
count=1`)
    .then(res => res.json())
    .then(data => {
      let carta = data.cards[0]

      // mostrar como ultima carta robada
      let ultimaCarta = document.getElementById('ultima-carta')
      ultimaCarta.innerHTML = '' // limpiar la anterior

      let img = document.createElement('img')
      img.src = carta.image
      img.alt = `${carta.value} de ${carta.suit}`
      img.style.width = '120px'
      ultimaCarta.appendChild(img)

      // añadir al historial de cartas robadas
      let historial = document.getElementById('historial')
      let imgHist = document.createElement('img')
      imgHist.src = carta.image
      imgHist.style.width = '60px'
      historial.appendChild(imgHist)

      // actualizar cartas restantes
      document.getElementById('restantes').textContent =
        `Cartas restantes: ${data.remaining}`
    })
})

```

Galeria de imagenes desde un array del JSON

Cuando el JSON devuelve un array de objetos con imagenes, recorres con `forEach` o `map` y creas una imagen por cada elemento:

```
// JSON con array de imagenes:
// [
//   { image: 'https://.../foto1.jpg', titulo: 'Playa' },
//   { image: 'https://.../foto2.jpg', titulo: 'Montana' },
// ]

fetch('data/fotos.json')
  .then(res => res.json())
  .then(fotos => {
    let contenedor = document.getElementById('galeria')
    contenedor.innerHTML = '' // limpiar antes de mostrar

    fotos.forEach(foto => {
      // crear un contenedor para cada imagen
      let div = document.createElement('div')

      let img = document.createElement('img')
      img.src = foto.image
      img.alt = foto.titulo
      img.style.width = '150px'
      img.style.margin = '5px'

      let titulo = document.createElement('p')
      titulo.textContent = foto.titulo

      div.appendChild(img)
      div.appendChild(titulo)
      contenedor.appendChild(div)
    })
  })
```

Con innerHTML (forma rapida)

Si no necesitas manipular cada imagen individualmente, `innerHTML` es mas rapido de escribir:

```
fetch('data/fotos.json')
  .then(res => res.json())
  .then(fotos => {
    let html = fotos.map(foto => `
      <div>
        <img src='${foto.image}' alt='${foto.titulo}'
style='width:150px' />
        <p>${foto.titulo}</p>
      </div>
    `).join('')
```

```
document.getElementById('galeria').innerHTML = html
})
```

Controlar errores de carga de imagen

A veces una URL de imagen falla. Puedes detectarlo con el evento `onerror` y poner una imagen de sustitucion:

```
let img = document.createElement('img')
img.src = foto.image

// si la imagen no carga, mostrar una de sustitucion
img.onerror = function() {
  img.src = 'img/sin-imagen.png'
  img.alt = 'Imagen no disponible'
}

// si la imagen carga bien
img.onload = function() {
  console.log('Imagen cargada:', img.src)
}
```

18. reduce() — el metodo mas potente de arrays

`reduce()` recorre un array y lo va 'reduciendo' a un unico valor acumulando resultados. Es el mas complejo pero tambien el mas potente.

La estructura es siempre:

```
array.reduce((acumulador, elemento) => {
  // acumulador: lo que llevas acumulado hasta ahora
  // elemento: el elemento actual del array
  return nuevoAcumulador
}, valorInicial)
// ^ esto es con lo que empieza el acumulador
```

Ejemplos de menor a mayor dificultad:


```
// sumar todos los numeros
let nums = [1, 2, 3, 4, 5]
let suma = nums.reduce((acc, n) => acc + n, 0) // 15

// encontrar el maximo
let max = nums.reduce((acc, n) => n > acc ? n : acc, 0) // 5

// sumar una propiedad de objetos
let productos = [{ precio: 10 }, { precio: 20 }, { precio: 5 }]
let total = productos.reduce((acc, p) => acc + p.precio, 0) // 35

// contar cuantas veces aparece cada letra
let letras = ['a', 'b', 'a', 'c', 'b', 'a']
let conteo = letras.reduce((acc, letra) => {
  acc[letra] = (acc[letra] || 0) + 1
  return acc
}, {})
// { a: 3, b: 2, c: 1 }
```

Resultado en consola:

```
suma    → 15
max     → 5
total   → 35
conteo  → { a: 3, b: 2, c: 1 }
```

Ejemplo del examen — normalizarDatos con reduce:

```
// entrada:
// [ {id:1, categorias:['A','B']}, {id:2, categorias:['A']}, ... ]
// salida deseada:
// { A: [1,2,3], B: [1], C: [3] }

function normalizarDatos(datos) {
  return datos.reduce((acc, item) => {

    item.categorias.forEach(cat => {
      // si la categoria no existe en acc, crearla como array vacio
      if (!acc[cat]) acc[cat] = []
      // anadir el id a esa categoria
      acc[cat].push(item.id)
    })

    return acc
  }, {}) // empieza con objeto vacio
}

let datos = [
  { id: 1, categorias: ['A', 'B'] },
  { id: 2, categorias: ['A'] },
  { id: 3, categorias: ['C', 'A'] }
]

console.log(normalizarDatos(datos))
// { A: [1,2,3], B: [1], C: [3] }
```

Resultado en consola:

```
{ A: [ 1, 2, 3 ], B: [ 1 ], C: [ 3 ] }
```

ComprimirOptimo con reduce (el otro ejercicio del examen):

```
function comprimirOptimo(texto) {
  return texto.split('').reduce((acc, letra) => {

    const ultimo = acc[acc.length - 1]

    // si el ultimo grupo es la misma letra, incrementar contador
    if (ultimo && ultimo.letra === letra) {
      ultimo.count++
    } else {
      // nueva letra, crear nuevo grupo
      acc.push({ letra, count: 1 })
    }

    return acc
  }, [])
  .map(g => g.letra + g.count) // convertir a 'a3', 'b1'...
  .join('')                  // unir todo
}

console.log(comprimirOptimo('aaabccdddde')) // 'a3b1c2d4e1'
```

Resultado en consola:

```
a3b1c2d4e1
```

20. POO avanzada

Propiedades y metodos privados con

En JS moderno puedes declarar propiedades y metodos privados con el simbolo #. Solo son accesibles desde dentro de la propia clase. Desde fuera dan error de sintaxis, ni siquiera con try/catch puedes acceder.

```
class CuentaBancaria {
  #saldo = 0 // privado: nadie de fuera puede tocarlo
  #titular // privado

  constructor(titular, saldoInicial) {
    this.#titular = titular
    this.#saldo = saldoInicial
  }

  depositar(cantidad) {
```

```

    if (cantidad <= 0) throw new Error('Cantidad invalida')
    this.#saldo += cantidad
  }

  retirar(cantidad) {
    if (cantidad > this.#saldo) throw new Error('Saldo insuficiente')
    this.#saldo -= cantidad
  }

  get saldo() { return this.#saldo }    // getter publico para leer

  info() { return `${this.#titular}: ${this.#saldo} euros` }
}

let cuenta = new CuentaBancaria('Ana', 1000)
cuenta.depositar(500)
console.log(cuenta.info())
cuenta.retirar(200)
console.log(cuenta.saldo)

// cuenta.#saldo          → ERROR DE SINTAXIS (ni compila)
// cuenta.#titular = 'X' → ERROR DE SINTAXIS

```

Resultado en consola:

```

Ana: 1500 euros
1300

```

Polimorfismo — mismo metodo, distinto comportamiento

El polimorfismo ocurre cuando varias clases hijas sobrescriben un metodo del padre con su propia implementacion. El padre define el contrato, cada hijo lo cumple a su manera. Lo importante: puedes tratar a todos los objetos igual aunque sean de tipos diferentes.

```

class Forma {
  area() { return 0 }
  describir() {
    return `Soy una forma con area ${this.area()}`
    // llama a area() de CADA clase hija, no a la del padre
  }
}

class Circulo extends Forma {
  constructor(radius) { super(); this.radius = radius }
  area() { return Math.round(Math.PI * this.radius ** 2) }
}

class Cuadrado extends Forma {
  constructor(lado) { super(); this.lado = lado }
  area() { return this.lado ** 2 }
}

// puedes meterlos todos en un array y tratarlos igual
let formas = [new Circulo(5), new Cuadrado(4), new Circulo(3)]

```

```
formas.forEach(f => console.log(f.describir()))
```

Resultado en consola:

```
Soy una forma con area 79
Soy una forma con area 16
Soy una forma con area 28
```

instanceof — comprobar de que clase es un objeto

instanceof comprueba si un objeto es instancia de una clase. También devuelve true para las clases padre (herencia incluida).

```
let c = new Circulo(5)

console.log(c instanceof Circulo)    // true
console.log(c instanceof Forma)      // true (hereda de Forma)
console.log(c instanceof Cuadrado)    // false

// util para saber que tipo de objeto tienes
formas.forEach(f => {
  if (f instanceof Circulo) {
    console.log(`Circulo de radio ${f.radio}`)
  } else if (f instanceof Cuadrado) {
    console.log(`Cuadrado de lado ${f.lado}`)
  }
})
```

Resultado en consola:

```
true
true
false
Circulo de radio 5
Cuadrado de lado 4
Circulo de radio 3
```

Clases abstractas simuladas

JavaScript no tiene clases abstractas nativas como Java o C#, pero se pueden simular lanzando un error si alguien intenta instanciar la clase base directamente. new.target devuelve la clase que se está instanciando.

```
class Vehiculo {
  constructor(marca) {
    // new.target es la clase que se llamo con new
    if (new.target === Vehiculo) {
      throw new Error('Vehiculo es abstracta, usa Coche o Moto')
    }
  }
}
```

```

    this.marca = marca
  }

  // metodo abstracto simulado: obliga a implementarlo
  arrancar() { throw new Error('arrancar() debe implementarse') }
  frenar()    { throw new Error('frenar() debe implementarse') }
}

class Coche extends Vehiculo {
  arrancar() { return `${this.marca} arranca: Brum brum` }
  frenar()   { return `${this.marca} frena` }
}

let c = new Coche('Toyota')
console.log(c.arrancar())
console.log(c.frenar())

try {
  let v = new Vehiculo('X') // lanza error
} catch(e) {
  console.log(e.message)
}

```

Resultado en consola:

```

Toyota arranca: Brum brum
Toyota frena
Vehiculo es abstracta, usa Coche o Moto

```

Mixins — añadir capacidades sin herencia

JS solo permite heredar de una clase. Los mixins son una forma de 'mezclar' capacidades de varias fuentes sin herencia multiple. Se implementan como funciones que reciben una clase y devuelven otra clase con nuevos metodos.

```

// mixin: funcion que recibe una clase y devuelve una clase ampliada
const Nadador = (Base) => class extends Base {
  nadar() { return `${this.nombre} esta nadando` }
}

const Corredor = (Base) => class extends Base {
  correr() { return `${this.nombre} esta corriendo` }
}

class Persona {
  constructor(nombre) { this.nombre = nombre }
}

// Triatleta hereda de Persona y tiene los dos mixins
class Triatleta extends Nadador(Corredor(Persona)) {}

let t = new Triatleta('Carlos')
console.log(t.nadar())
console.log(t.correr())
console.log(t instanceof Persona) // true

```

Resultado en consola:
Carlos esta nadando
Carlos esta corriendo
true

Validar valores permitidos en el constructor

Una tecnica muy habitual es restringir que una propiedad solo pueda tomar ciertos valores. Se valida en el constructor (y en el setter si existe) comparando contra un array o constante de valores validos.

Forma basica — validar en el constructor:

```
class Dinero {
  static MONEDAS_VALIDAS = ['euros', 'dolares', 'pesos']

  constructor(cantidad, moneda) {
    if (!Dinero.MONEDAS_VALIDAS.includes(moneda)) {
      throw new Error(
        `Moneda '${moneda}' no valida. Usa: $
{Dinero.MONEDAS_VALIDAS.join(', ')}`
      )
    }
    if (cantidad < 0) throw new Error('La cantidad no puede ser
negativa')
    this.cantidad = cantidad
    this.moneda = moneda
  }

  info() { return `${this.cantidad} ${this.moneda}` }
}

let d1 = new Dinero(100, 'euros')
console.log(d1.info())

try {
  let d2 = new Dinero(100, 'yenes') // no esta en la lista
} catch(e) {
  console.log(e.message)
}
```

Resultado en consola:
100 euros
Moneda 'yenes' no valida. Usa: euros, dolares, pesos

Forma avanzada — validar tambien al cambiar el valor con setter:

```

class Producto {
  static CATEGORIAS = ['electronica', 'ropa', 'alimentacion']
  #categoria

  constructor(nombre, categoria) {
    this.nombre = nombre
    this.categoria = categoria // pasa por el setter
  }

  get categoria() { return this.#categoria }

  set categoria(valor) {
    if (!Producto.CATEGORIAS.includes(valor)) {
      throw new Error(
        `Categoria '${valor}' no valida. Opciones: $
        {Producto.CATEGORIAS.join(', ')}`
      )
    }
    this.#categoria = valor
  }

  info() { return `${this.nombre} (${this.#categoria})` }
}

let p1 = new Producto('Camisa', 'ropa')
console.log(p1.info())

try {
  p1.categoria = 'juguetes' // intento de cambiar a invalido
} catch(e) {
  console.log(e.message)
}

console.log(p1.info()) // sigue siendo 'ropa', no cambio

```

Resultado en consola:

```

Camisa (ropa)
Categoria 'juguetes' no valida. Opciones: electronica, ropa,
alimentacion
Camisa (ropa)

```

Enums simulados con Object.freeze() — lista de constantes que no se pueden modificar:

```

// Object.freeze() impide añadir, modificar o eliminar propiedades
const Estado = Object.freeze({
  PENDIENTE: 'pendiente',
  ACTIVO: 'activo',
  CERRADO: 'cerrado'
})

class Pedido {
  #estado

```

```

constructor(id) {
  this.id = id
  this.#estado = Estado.PENDIENTE // siempre empieza pendiente
}

get estado() { return this.#estado }

cambiarEstado(nuevoEstado) {
  const validos = Object.values(Estado)
  if (!validos.includes(nuevoEstado)) {
    throw new Error(`Estado '${nuevoEstado}' no valido. Usa: $
{validos.join(', ')}`)
  }
  this.#estado = nuevoEstado
}

info() { return `Pedido #${this.id}: ${this.#estado}` }
}

let pedido = new Pedido(1)
console.log(pedido.info())
pedido.cambiarEstado(Estado.ACTIVO)
console.log(pedido.info())
pedido.cambiarEstado('cerrado') // tambien funciona con el string
console.log(pedido.info())

try {
  pedido.cambiarEstado('cancelado')
} catch(e) {
  console.log(e.message)
}

// freeze impide modificar el enum
Estado.NUEVO = 'nuevo'
console.log(Estado.NUEVO) // undefined, no se pudo añadir

```

Resultado en consola:

```

Pedido #1: pendiente
Pedido #1: activo
Pedido #1: cerrado
Estado 'cancelado' no valido. Usa: pendiente, activo, cerrado
undefined

```

21. Sintaxis moderna

Parametros por defecto (default parameters)

Puedes asignar un valor por defecto a los parametros de una funcion. Si no se pasa ese argumento, usa el valor por defecto.

```
function saludar(nombre = 'desconocido', idioma = 'es') {
```



```

    if (idioma === 'es') return `Hola, ${nombre}`
    if (idioma === 'en') return `Hello, ${nombre}`
  }

  console.log(saludar())           // usa los dos defaults
  console.log(saludar('Ana'))      // solo usa el segundo default
  console.log(saludar('Ana', 'en')) // sin defaults

  // tambien en clases:
  class Producto {
    constructor(nombre, precio = 0, stock = 1) {
      this.nombre = nombre
      this.precio = precio
      this.stock = stock
    }
  }
  let p = new Producto('Camisa')   // precio=0, stock=1

```

Resultado en consola:

```

Hola, desconocido
Hola, Ana
Hello, Ana

```

Destructuring — extraer valores de arrays y objetos

El destructuring permite extraer valores de arrays u objetos y asignarlos a variables en una sola linea. Es muy habitual en exámenes con datos de APIs.

```

// destructuring de ARRAY
let [primero, segundo, ...resto] = [1, 2, 3, 4, 5]
console.log(primero)    // 1
console.log(segundo)    // 2
console.log(resto)      // [3, 4, 5]

// destructuring de OBJETO
let { nombre, edad, ciudad = 'desconocida' } = { nombre: 'Luis',
edad: 30 }
console.log(nombre)     // 'Luis'
console.log(edad)       // 30
console.log(ciudad)     // 'desconocida' (valor por defecto)

// renombrar al desestructurar
let { nombre: n, edad: e } = { nombre: 'Ana', edad: 25 }
console.log(n, e)       // 'Ana' 25

// en parametros de funcion (muy util con APIs)
function mostrar({ nombre, edad = 0 }) {
  return `${nombre} tiene ${edad} anos`
}
console.log(mostrar({ nombre: 'Marta', edad: 22 }))
console.log(mostrar({ nombre: 'Marta' })) // edad usa default

```

Resultado en consola:

```

1

```

```
2
[ 3, 4, 5 ]
Luis
30
desconocida
Ana 25
Marta tiene 22 años
Marta tiene 0 años
```

Spread (...) y Rest (...)

El operador ... hace dos cosas segun donde se use: spread expande un array u objeto, rest recoge el resto de argumentos.

```
// SPREAD – expandir

// copiar y combinar arrays
let arr1 = [1, 2, 3]
let arr2 = [4, 5, 6]
let combinado = [...arr1, ...arr2]
console.log(combinado) // [1, 2, 3, 4, 5, 6]

// copia de array (sin referencias compartidas)
let copia = [...arr1]

// combinar objetos
let base = { a: 1, b: 2 }
let extendido = { ...base, c: 3 }
console.log(extendido) // { a: 1, b: 2, c: 3 }

// pasar array como argumentos
let nums = [3, 1, 4, 1, 5]
console.log(Math.max(...nums)) // 5

// REST – recoger el resto

// en funciones: recoge argumentos sobrantes
function sumarTodos(...numeros) {
  return numeros.reduce((acc, n) => acc + n, 0)
}
console.log(sumarTodos(1, 2, 3, 4, 5)) // 15

// en destructuring: recoge el resto
let [primero, ...losOtros] = [10, 20, 30, 40]
console.log(primero) // 10
console.log(losOtros) // [20, 30, 40]
```

Resultado en consola:

```
[ 1, 2, 3, 4, 5, 6 ]
{ a: 1, b: 2, c: 3 }
5
15
10
[ 20, 30, 40 ]
```

22. Manejo de errores — try / catch / finally / throw

El manejo de errores permite que el programa no se rompa cuando algo falla. Es imprescindible al trabajar con fetch, fechas, o cualquier operacion que pueda fallar.

```
// try: codigo que puede fallar
// catch: que hacer si falla
// finally: se ejecuta SIEMPRE, haya error o no

function dividir(a, b) {
  if (b === 0) throw new Error('No se puede dividir por cero')
  return a / b
}

try {
  console.log(dividir(10, 2))    // va bien
  console.log(dividir(10, 0))    // lanza error
  console.log('esto no llega')  // no se ejecuta
} catch(e) {
  console.log('Error:', e.message)
} finally {
  console.log('Esto siempre se ejecuta')
}
```

Resultado en consola:

```
5
Error: No se puede dividir por cero
Esto siempre se ejecuta
```

Tipos de error que puedes lanzar con throw:

```
throw new Error('mensaje generico')
throw new TypeError('tipo de dato incorrecto')
throw new RangeError('valor fuera de rango')

// en catch puedes ver el tipo:
try {
  throw new TypeError('esto no es un numero')
} catch(e) {
  console.log(e.name)      // 'TypeError'
  console.log(e.message)  // 'esto no es un numero'
}
```

Resultado en consola:

```
TypeError  
esto no es un numero
```

Patron tipico con fetch y async/await:

```
async function cargarDatos(url) {  
  try {  
    const res = await fetch(url)  
    if (!res.ok) throw new Error(`HTTP ${res.status}`)  
    const data = await res.json()  
    return data  
  } catch(e) {  
    console.error('Fallo la peticion:', e.message)  
    return null // devuelve null en vez de romper todo  
  }  
}
```

23. Operadores modernos — ?. ?? ||

Optional chaining (?.) — acceder sin errores

El operador ?. accede a una propiedad o llama a un metodo solo si el objeto existe. Si es null o undefined, devuelve undefined en vez de lanzar un error.

```
let u1 = { nombre: 'Ana', direccion: { ciudad: 'Madrid' } }  
let u2 = { nombre: 'Luis' } // sin direccion  
  
// sin ?. → error si direccion no existe  
// console.log(u2.direccion.ciudad) // TypeError!  
  
// con ?. → devuelve undefined si algo es null/undefined  
console.log(u1?.direccion?.ciudad) // 'Madrid'  
console.log(u2?.direccion?.ciudad) // undefined (sin error)  
  
// tambien funciona con metodos  
let arr = null  
console.log(arr?.length) // undefined (sin error)  
console.log(arr?.map(x=>x)) // undefined (sin error)
```

Resultado en consola:

```
Madrid  
undefined  
undefined  
undefined
```

Nullish coalescing (??) vs OR (||)

?? devuelve el valor de la derecha SOLO si el de la izquierda es null o undefined. || devuelve el de la derecha si el de la izquierda es cualquier valor falsy (0, "", false, null, undefined).

```
let valor = null
console.log(valor ?? 'por defecto') // 'por defecto'
console.log(valor || 'por defecto') // 'por defecto'

// la diferencia se ve con 0 o ''
let cero = 0
console.log(cero ?? 'por defecto') // 0 (0 no es
null/undefined)
console.log(cero || 'por defecto') // 'por defecto' (0 es falsy)

let texto = ''
console.log(texto ?? 'vacio') // '' ('' no es
null/undefined)
console.log(texto || 'vacio') // 'vacio' ('' es falsy)

// combinacion tipica en exámenes:
let datos = JSON.parse(localStorage.getItem('lista') ?? '[]')
// ?? '[]' solo actua si getItem devuelve null (clave no existe)
```

Resultado en consola:

```
por defecto
por defecto
0
por defecto
''
vacio
```

24. Map y Set — colecciones avanzadas

Map — diccionario clave/valor mejorado

Map es como un objeto pero con superpoderes: las claves pueden ser de cualquier tipo (no solo strings), mantiene el orden de insercion, y tiene metodos utiles.

```
let mapa = new Map()

// añadir entradas
mapa.set('nombre', 'Ana')
mapa.set('edad', 25)
mapa.set(42, 'una clave numerica') // las claves pueden ser numeros

// leer
console.log(mapa.get('nombre')) // 'Ana'
```

```

console.log(mapa.has('edad'))    // true
console.log(mapa.size)          // 3

// recorrer
mapa.forEach((valor, clave) => {
  console.log(`${clave}: ${valor}`)
})

// eliminar
mapa.delete('edad')
console.log(mapa.size)    // 2

// convertir a array
let entradas = [...mapa.entries()]
let claves   = [...mapa.keys()]
let valores  = [...mapa.values()]

```

Resultado en consola:

```

Ana
true
3
nombre: Ana
edad: 25
42: una clave numerica
2

```

Set — lista sin duplicados

Set es una coleccion de valores unicos. Si intentas añadir un valor que ya existe, lo ignora. Muy util para eliminar duplicados de un array.

```

// crea un Set (elimina duplicados automaticamente)
let conjunto = new Set([1, 2, 2, 3, 3, 3])
console.log(conjunto.size)    // 3 (solo valores unicos)

// añadir y comprobar
conjunto.add(4)
console.log(conjunto.has(2))  // true
console.log(conjunto.has(9))  // false

// convertir a array
console.log([...conjunto])    // [1, 2, 3, 4]

// TRUCO: eliminar duplicados de un array
let conDuplicados = [1, 2, 2, 3, 3, 4, 'a', 'a']
let sinDuplicados = [...new Set(conDuplicados)]
console.log(sinDuplicados)    // [1, 2, 3, 4, 'a']

// recorrer
conjunto.forEach(valor => console.log(valor))

```

Resultado en consola:

```

3
true

```

```
false  
[ 1, 2, 3, 4 ]  
[ 1, 2, 3, 4, 'a' ]
```

25. Object — metodos utiles

El objeto Object tiene metodos estaticos muy utiles para trabajar con objetos. Los mas habituales en exámenes son keys, values, entries y assign.

```
let persona = { nombre: 'Ana', edad: 25, ciudad: 'Madrid' }  
  
// Object.keys() → array con las claves  
console.log(Object.keys(persona))  
// ['nombre', 'edad', 'ciudad']  
  
// Object.values() → array con los valores  
console.log(Object.values(persona))  
// ['Ana', 25, 'Madrid']  
  
// Object.entries() → array de pares [clave, valor]  
console.log(Object.entries(persona))  
// [['nombre', 'Ana'], ['edad', 25], ['ciudad', 'Madrid']]  
  
// recorrer objeto con entries (muy util)  
Object.entries(persona).forEach(([clave, valor]) => {  
  console.log(`${clave}: ${valor}`)  
})  
  
// Object.assign() → copiar/mezclar objetos  
let extra = { profesion: 'ingeniera', edad: 26 }  
let completo = Object.assign({}, persona, extra)  
console.log(completo)  
// edad queda como 26 (extra sobrescribe persona)  
  
// alternativa moderna con spread (hace lo mismo)  
let completo2 = { ...persona, ...extra }
```

Resultado en consola:

```
[ 'nombre', 'edad', 'ciudad' ]  
[ 'Ana', 25, 'Madrid' ]  
[ [ 'nombre', 'Ana' ], [ 'edad', 25 ], [ 'ciudad', 'Madrid' ] ]  
nombre: Ana  
edad: 25  
ciudad: Madrid  
{ nombre: 'Ana', edad: 26, ciudad: 'Madrid', profesion: 'ingeniera' }
```

Convertir un objeto a array y viceversa:

```
// objeto → array de entradas → filtrar → volver a objeto
let precios = { manzana: 1.2, pera: 0.8, uva: 2.5, kiwi: 3.0 }

// filtrar los que cuestan mas de 1 euro
let caros = Object.fromEntries(
  Object.entries(precios).filter(([k, v]) => v > 1)
)
console.log(caros)
// { manzana: 1.2, uva: 2.5, kiwi: 3 }
```

Resultado en consola:
{ manzana: 1.2, uva: 2.5, kiwi: 3 }

19. Errores comunes

⚠ = en vez de === en comparaciones

Un solo = asigna valor. Tres === compara. En un if siempre usa ===.

⚠ Acceder a un elemento que no existe

Si getElementById no encuentra el elemento devuelve null, y cualquier cosa que hagas con null da error. Comprueba que el id del HTML y del JS coinciden exactamente.

⚠ Olvidar .value en los inputs

Para leer lo que el usuario ha escrito necesitas input.value. Sin .value obtienes el elemento HTML, no el texto.

⚠ Los índices del array empiezan en 0

Si tienes 3 elementos, los índices son 0, 1 y 2. array[3] devuelve undefined, no el cuarto elemento.

⚠ Abrir el HTML con doble click al usar fetch

fetch no funciona abriendo el fichero directamente desde el explorador. Necesitas un servidor local como la extensión Live Server de VS Code.

26. HTML de referencia

Referencia rápida de todo el HTML que necesitas para los exámenes. No hace falta memorizar todo, pero si tenerlo a mano.

Estructura base de cualquier pagina

Esta es la plantilla minima que va en todo fichero HTML. En VS Code escribe ! y pulsa Tab para generarla automaticamente.

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Titulo de la pagina</title>
    <!-- CSS externo (opcional) -->
    <link rel="stylesheet" href="estilos.css" />
  </head>
  <body>

    <!-- todo el contenido visible va aqui -->

    <!-- JS al final del body para que el HTML cargue primero -->
    <script src="script.js"></script>
    <!-- o JS inline: -->
    <script>
      console.log('Hola')
    </script>
  </body>
</html>
```

Etiquetas de texto y estructura

```
<!-- Titulos: h1 es el mas grande, h6 el mas pequeño -->
<h1>Titulo principal</h1>
<h2>Subtitulo</h2>
<h3>Sub-subtitulo</h3>

<!-- Parrafo -->
<p>Esto es un parrafo de texto normal.</p>

<!-- Texto en negrita e italica -->
<strong>Negrita</strong>
<em>Italica</em>

<!-- Salto de linea -->
<br />

<!-- Linea horizontal -->
<hr />

<!-- Enlace -->
<a href="https://google.com">Ir a Google</a>
<a href="otra-pagina.html">Pagina interna</a>
```

```
<!-- Span: contenedor inline (no ocupa linea entera) -->
<span id="resultado">texto</span>

<!-- Div: contenedor bloque (ocupa linea entera) -->
<div id="contenedor"></div>
```

Inputs y formularios

Los inputs son la forma de recibir datos del usuario. Cada tipo tiene un comportamiento distinto.

```
<!-- Texto libre -->
<input type="text" id="nombre" placeholder="Escribe aqui" />

<!-- Solo numeros -->
<input type="number" id="edad" min="0" max="120" value="18" />

<!-- Contraseña (oculta los caracteres) -->
<input type="password" id="pass" />

<!-- Casilla de verificación -->
<input type="checkbox" id="activo" />
<label for="activo">Esta activo</label>

<!-- Opciones circulares (solo una seleccionable) -->
<input type="radio" name="color" value="rojo" /> Rojo
<input type="radio" name="color" value="azul" /> Azul

<!-- Lista desplegable -->
<select id="pais">
  <option value="">-- Selecciona --</option>
  <option value="es">España</option>
  <option value="mx">Mexico</option>
</select>

<!-- Texto multilínea -->
<textarea id="comentario" rows="4" cols="40"></textarea>

<!-- Boton -->
<button id="btn">Pulsar</button>
<button type="submit">Enviar</button>

<!-- Leer valores desde JS -->
document.getElementById('nombre').value      // texto
document.getElementById('edad').value        // string! usar
parseInt()
document.getElementById('activo').checked    // true/false
document.getElementById('pais').value        // valor del option
seleccionado
```

Tablas

Las tablas tienen tres partes: thead (cabecera), tbody (cuerpo) y opcionalmentetfoot (pie). Desde JS casi siempre se generan dinámicamente.

```
<!-- tabla estatica en HTML -->
<table id="miTabla">
  <thead>
    <tr>
      <th>Nombre</th>
      <th>Edad</th>
      <th>Ciudad</th>
    </tr>
  </thead>
  <tbody id="cuerpo-tabla">
    <!-- las filas se generan desde JS -->
  </tbody>
</table>

<!-- generar filas desde JS con innerHTML -->
let datos = [{nombre:'Ana', edad:25}, {nombre:'Luis', edad:30}]
let tbody = document.getElementById('cuerpo-tabla')
tbody.innerHTML = '' // limpiar antes de rellenar
datos.forEach(d => {
  tbody.innerHTML +=
`<tr><td>${d.nombre}</td><td>${d.edad}</td></tr>`
})
```

Listas

```
<!-- lista con puntos (unordered list) -->
<ul id="mi-lista">
  <li>Elemento 1</li>
  <li>Elemento 2</li>
</ul>

<!-- lista numerada (ordered list) -->
<ol>
  <li>Primero</li>
  <li>Segundo</li>
</ol>

<!-- generar lista desde JS -->
let frutas = ['Manzana', 'Pera', 'Naranja']
let ul = document.getElementById('mi-lista')
ul.innerHTML = frutas.map(f => `<li>${f}</li>`).join('')
```

Imágenes

```
<!-- imagen estatica -->


<!-- imagen desde JS -->
let img = document.createElement('img')
img.src = 'https://ejemplo.com/foto.jpg'
img.alt = 'Descripcion'
img.style.width = '200px'
document.getElementById('contenedor').appendChild(img)

<!-- imagen con control de error -->
img.onerror = () => { img.src = 'sin-imagen.png' }
```

Estilos inline desde JS

Puedes cambiar el aspecto de cualquier elemento directamente desde JS sin tocar el CSS.

```
let elem = document.getElementById('caja')

elem.style.color = 'red'
elem.style.backgroundColor = '#FFCC00' // camelCase, no background-color
elem.style.fontSize = '20px'
elem.style.fontWeight = 'bold'
elem.style.display = 'none' // ocultar
elem.style.display = 'block' // mostrar
elem.style.border = '1px solid black'
elem.style.padding = '10px'
elem.style.margin = '5px'
elem.style.width = '200px'
elem.style.textAlign = 'center'

// añadir/quitar clases CSS
elem.classList.add('activo')
elem.classList.remove('activo')
elem.classList.toggle('activo') // si tiene la clase la quita, si no la pone
```

Manipulación del DOM — referencia rápida

```
// SELECCIONAR elementos
document.getElementById('id') // por id (uno)
```

```

document.querySelector('.clase')           // por clase CSS (primero)
document.querySelector('#id')              // por id (igual que
getElementById)
document.querySelectorAll('p')            // todos los <p>
(NodeList)
document.querySelectorAll('.tarjeta')     // todos con clase
'tarjeta'

// LEER Y ESCRIBIR contenido
elem.textContent = 'texto plano'           // sin HTML
elem.innerHTML   = '<b>con HTML</b>'       // con HTML
elem.value       // valor de input/select

// CREAR e INSERTAR elementos
let div = document.createElement('div')
div.textContent = 'Nuevo elemento'
padre.appendChild(div)                    // insertar al final
padre.prepend(div)                        // insertar al principio
padre.removeChild(div)                    // eliminar hijo

// ATRIBUTOS
elem.setAttribute('src', 'foto.jpg')
elem.getAttribute('src')
elem.removeAttribute('disabled')

// EVENTOS mas usados
elem.addEventListener('click', () => {})
elem.addEventListener('input', () => {})  // al escribir en input
elem.addEventListener('change', () => {}) // al cambiar
select/checkbox
elem.addEventListener('keydown', (e) => { console.log(e.key) })
window.onload = function() {}             // al cargar la pagina

```

Estructura tipica de examen — HTML completo

Este es el patron que se repite en casi todos los exámenes: input + boton + div de resultado + tabla.

```

<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Ejercicio</title>
  </head>
  <body>
    <h1>Titulo del ejercicio</h1>

    <!-- entrada del usuario -->
    <input type="text" id="entrada" placeholder="Escribe algo" />
    <button id="btn">Buscar</button>

    <!-- zona donde se muestra el resultado -->
    <div id="resultado"></div>

    <!-- tabla para el historial -->

```

```

<h2>Historial</h2>
<table>
  <thead>
    <tr><th>Columna 1</th><th>Columna 2</th></tr>
  </thead>
  <tbody id="tabla-body"></tbody>
</table>

<script>
  const btn      = document.getElementById('btn')
  const entrada  = document.getElementById('entrada')
  const resultado = document.getElementById('resultado')
  const tablaBody = document.getElementById('tabla-body')

  btn.addEventListener('click', () => {
    let valor = entrada.value.trim()
    if (!valor) { resultado.innerHTML = '<p>Escribe algo</p>';
return }

    fetch(`https://api.ejemplo.com/${valor}`)
      .then(res => res.json())
      .then(data => {
        resultado.innerHTML = `<p>${data.campo}</p>`
        tablaBody.innerHTML += `<tr><td>${valor}</td><td>${
{data.campo}</td></tr>`
      })
      .catch(() => { resultado.innerHTML = '<p>Error</p>' })
    })

    window.onload = () => { /* cargar datos guardados */ }
  </script>
</body>
</html>

```

27. Extensiones de VS Code para el examen

Estas son las extensiones que debes instalar ANTES de que corten internet. Una vez instaladas funcionan sin conexión. Para instalar: abre VS Code, pulsa Ctrl+Shift+X y busca el nombre exacto.

Extension	Autor	Para que sirve (todas funcionan SIN internet)
Live Server	Ritwick Dey	IMPRESINDIBLE. Servidor local para abrir HTML. Click derecho en el .html → Open with Live Server. Sin esto el fetch no funciona.
Prettier	Prettier	Formatea el código automáticamente al guardar (Ctrl+S). Evita errores de indentación y llaves mal cerradas.
JavaScript (ES6) snippets	xabikos	Atajos: clg+Tab → console.log(), fn+Tab → function, afn+Tab → función flecha,

		fre+Tab → forEach, etc.
HTML CSS Support	ecmel	Autocompleta clases CSS dentro del HTML. Sugiere nombres de clases al escribir class="".
IntelliSense for CSS class names	Zignd	Escanea tus ficheros CSS y sugiere los nombres de clase disponibles al escribir en el HTML.
Auto Close Tag	Jun Han	Cierra automaticamente las etiquetas HTML al escribir >. Escribe <div y se añade </div> solo.
Auto Rename Tag	Jun Han	Al cambiar una etiqueta de apertura (<div> a <section>) cambia automaticamente la de cierre.
Path Intellisense	Christian Kohler	Autocompleta rutas de ficheros al escribir src=" o href=". Evita errores de ruta al enlazar JSON, CSS o JS.
indent-rainbow	oderwat	Colorea los niveles de indentacion con colores distintos. Facilita ver la estructura del codigo anidado.

Atajos de teclado imprescindibles en VS Code

Atajo	Que hace
! + Tab	Genera la estructura HTML base completa
Ctrl + S	Guardar (con Prettier activo, tambien formatea)
Ctrl + Z	Deshacer
Ctrl + Shift + Z	Rehacer
Ctrl + D	Seleccionar la siguiente ocurrencia de la palabra seleccionada
Ctrl + H	Buscar y reemplazar en el fichero
Ctrl + /	Comentar/descomentar la linea actual
Alt + Flecha arriba/abajo	Mover la linea actual hacia arriba o abajo
Ctrl + L	Seleccionar la linea completa
Ctrl + `	Abrir/cerrar la terminal integrada
F12	Ir a la definicion de una funcion o variable
Ctrl + P	Buscar y abrir un fichero del proyecto
Ctrl + Shift + P	Abrir la paleta de comandos (todo lo que puede hacer VS Code)
Ctrl + B	Mostrar/ocultar el explorador de ficheros lateral
Alt + Click	Añadir varios cursores a la vez para editar en varios sitios

Snippets utiles con JavaScript (ES6) snippets

Con la extension JavaScript (ES6) snippets instalada, estos atajos te ahorran tiempo:

Escribe	Genera
clg	<code>console.log()</code>
fn	<code>function nombreFuncion(params) {}</code>
afn	<code>const nombre = (params) => {}</code> (funcion flecha)
imp	<code>import ... from '...'</code>
imd	<code>const { } = require('')</code>
nfn	<code>const nombre = function(params) {}</code>
dob	<code>const { } = objeto</code> (destructuring)
dar	<code>const [] = array</code> (destructuring)
fre	<code>array.forEach((elemento) => {})</code>
fof	<code>for (const item of array) {}</code>
sti	<code>setTimeout(() => {}, milisegundos)</code>
pto	Promise con then y catch

Plan de accion antes del examen

Pasos a seguir ANTES de que corten internet. Una vez instaladas, todas funcionan sin conexion:

- 1. Abrir VS Code → Ctrl+Shift+X → buscar e instalar cada extension por su nombre exacto
- 2. Extensiones a instalar: Live Server (Ritwick Dey), Prettier, JavaScript (ES6) snippets (xabikos), HTML CSS Support (ecmel), IntelliSense for CSS class names (Zignd), Auto Close Tag, Auto Rename Tag, Path Intellisense, indent-rainbow
- 3. Configurar Prettier para que formatee al guardar: Ctrl+Shift+P → escribir 'Open User Settings JSON' → añadir la linea: "editor.formatOnSave": true
- 4. Verificar Live Server: crear un fichero index.html de prueba → click derecho sobre el fichero → 'Open with Live Server' → debe abrirse el navegador
- 5. Verificar snippets: abrir un .js → escribir clg + Tab → debe expandirse a console.log()

- 6. Tener este documento abierto o impreso para consultar durante el examen

28. Resumen rapido

Concepto	Que es	Ejemplo corto
let / const	Guardar datos	let x = 5
if / else	Decidir segun condicion	if (x > 3) { ... }
function	Bloque de codigo con nombre	function sumar(a,b) { return a+b }
array []	Lista de elementos	let lista = [1, 2, 3]
objeto {}	Datos con clave:valor	let p = { nombre: 'Ana' }
getElementById	Seleccionar elemento HTML	document.getElementById('id')
addEventListener	Reaccionar a eventos	btn.addEventListener('click', ...)
innerHTML	Cambiar HTML de un elemento	div.innerHTML = ' Hola '
.value	Leer valor de un input	input.value
fetch()	Pedir datos a un fichero/API	fetch('datos.json').then(...)
find()	Buscar en un array	array.find(x => x.id === 1)
filter()	Filtrar array	array.filter(x => x.edad > 18)
forEach()	Recorrer array	array.forEach(x => console.log(x))
class	Crear una clase (molde de objetos)	class Persona { constructor() {} }
new	Crear un objeto de una clase	let p = new Persona('Ana', 25)
this	Referenciar el propio objeto	this.nombre = nombre
super()	Llamar al constructor padre	super(nombre)
#propiedad	Propiedad privada (solo dentro de la clase)	class C { #x = 0 }
instanceof	Comprobar tipo de objeto	obj instanceof MiClase
new.target	Clase que se esta instanciando	if (new.target === X) throw
Mixin	Anadir capacidades sin herencia multiple	const M = (B) => class extends B {}
param = valor	Parametro con valor por defecto	function f(x = 0) {}
{ a, b } = obj	Destructuring de objeto	let { nombre, edad } = persona
[a, b] = arr	Destructuring de array	let [x, y, ...r] = [1,2,3,4]
...spread	Expandir array u objeto	[...arr1, ...arr2]

...rest	Recoger argumentos sobrantes	function f(...args) {}
try/catch/finally	Manejar errores	try {} catch(e) {} finally {}
throw	Lanzar un error	throw new Error('mensaje')
?.	Acceder sin error si es null	obj?.prop?.sub
??	Valor por defecto solo si null/undefined	val ?? 'default'
Map	Diccionario clave/valor avanzado	new Map(); map.set/get/has
Set	Lista sin duplicados	new Set([1,2,2,3]) → {1,2,3}
Object.keys()	Array de claves del objeto	Object.keys({a:1}) → ['a']
Object.values()	Array de valores del objeto	Object.values({a:1}) → [1]
Object.entries()	Array de pares [clave,valor]	Object.entries({a:1})
Object.assign()	Mezclar objetos	Object.assign({}, obj1, obj2)
async/await	Fetch mas legible	async function f() { await fetch() }
XMLHttpRequest	Peticion HTTP clasica	xhr.open(); xhr.onload=...; xhr.send()
JSON.stringify()	Objeto/array a texto JSON	JSON.stringify({a:1})
JSON.parse()	Texto JSON a objeto/array	JSON.parse('{"a":1}')
localStorage.setItem	Guardar en el navegador	localStorage.setItem('k', valor)
localStorage.getItem	Leer del navegador	localStorage.getItem('k')
reduce()	Reducir array a un valor	array.reduce((acc,x) => acc+x, 0)
Date	Trabajar con fechas	new Date(ano, mes-1, dia)
split()	Dividir string en array	'a,b'.split(',') → ['a','b']
createElement	Crear elemento HTML desde JS	document.createElement('td')
appendChild	Insertar elemento en el DOM	padre.appendChild(hijo)

Recuerda: la mejor forma de aprender a programar es practicando. Prueba, rompe cosas y corrégelas.